TP1-TD1 JEE

Introduction au TP JEE :

Le **Java Enterprise Edition (JEE)** est une plateforme de développement qui permet de créer des applications web et des applications d'entreprise robustes et évolutives. Contrairement à Java SE (Standard Edition), JEE fournit des **API et services supplémentaires** pour gérer les aspects réseau, la persistance des données, la sécurité, et l'interaction avec des serveurs d'applications.

L'objectif : de ce TP est de **découvrir les composants essentiels de JEE**, tels que :

- **Servlets** : pour gérer les requêtes HTTP et générer des réponses dynamiques.

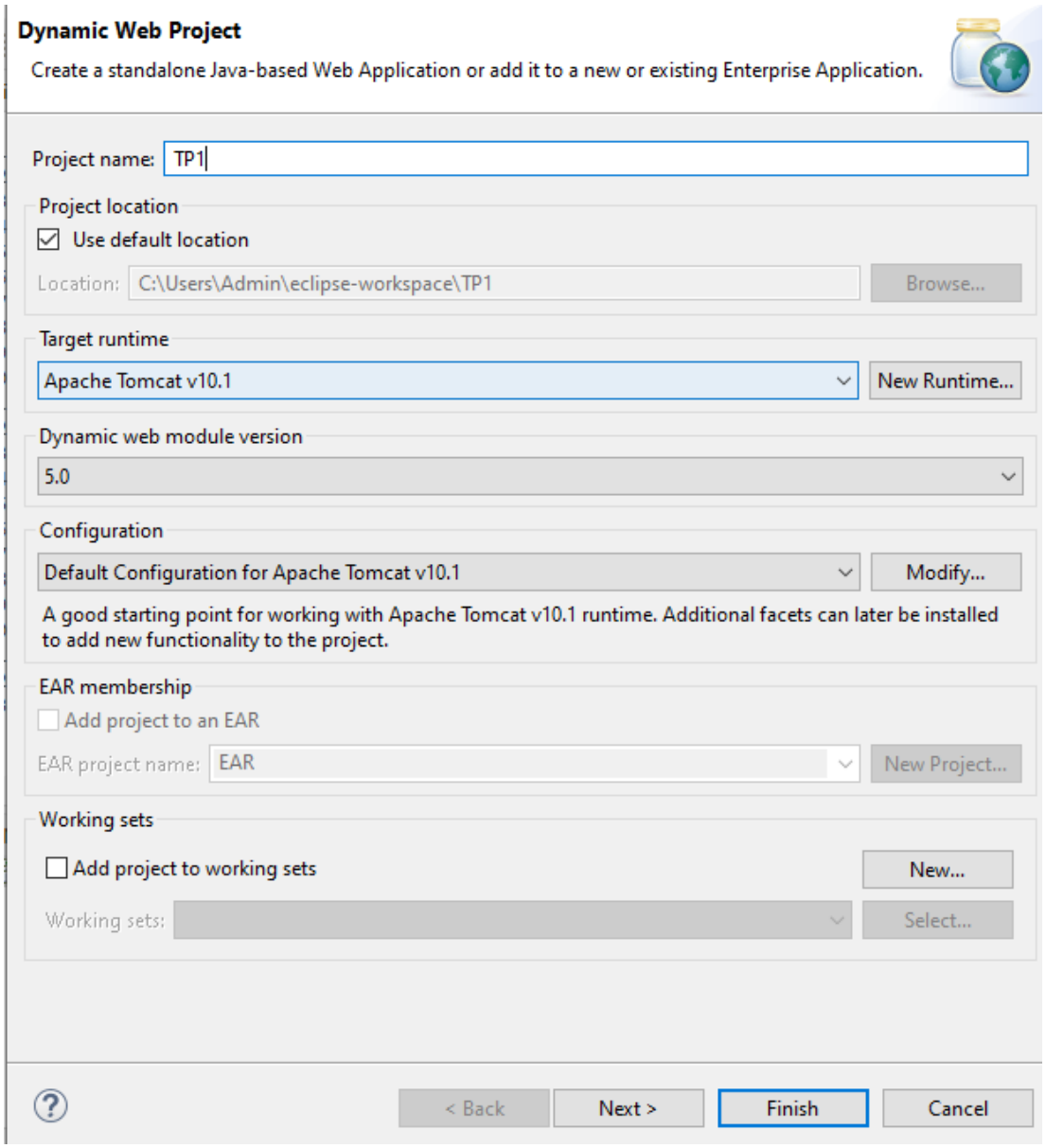
À travers ce TP, vous allez :

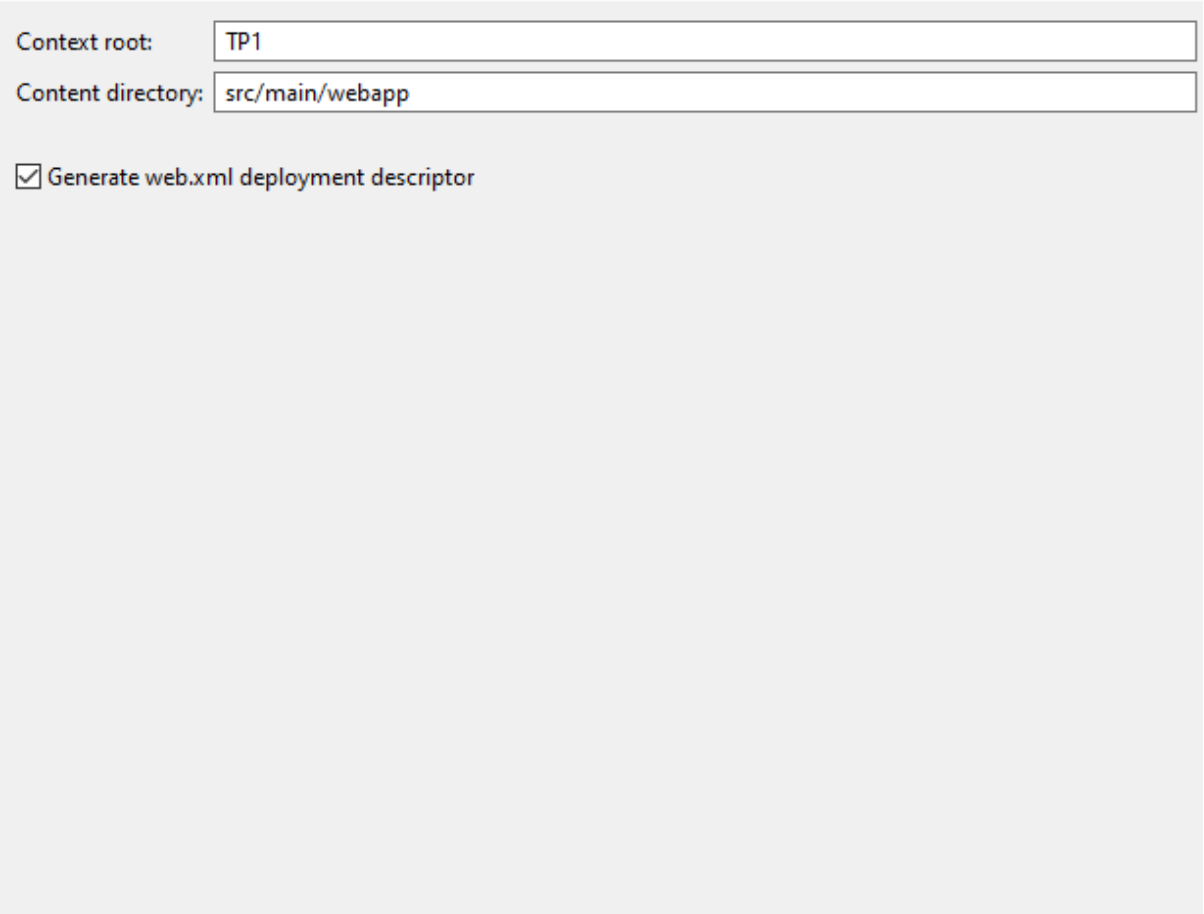
1. Créer un **formulaire web** pour saisir des informations.
2. Développer une **Servlet** qui récupère ces données et les traite.
3. Afficher le résultat dans **une page Html ou dans la réponse de la Servlet.**

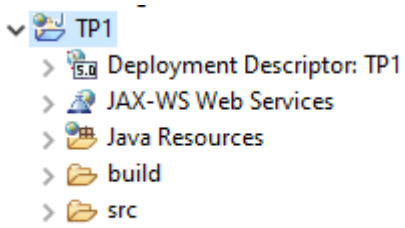
Technologies utilisées[1](voir la fin de document)

[1]

Créations de projet



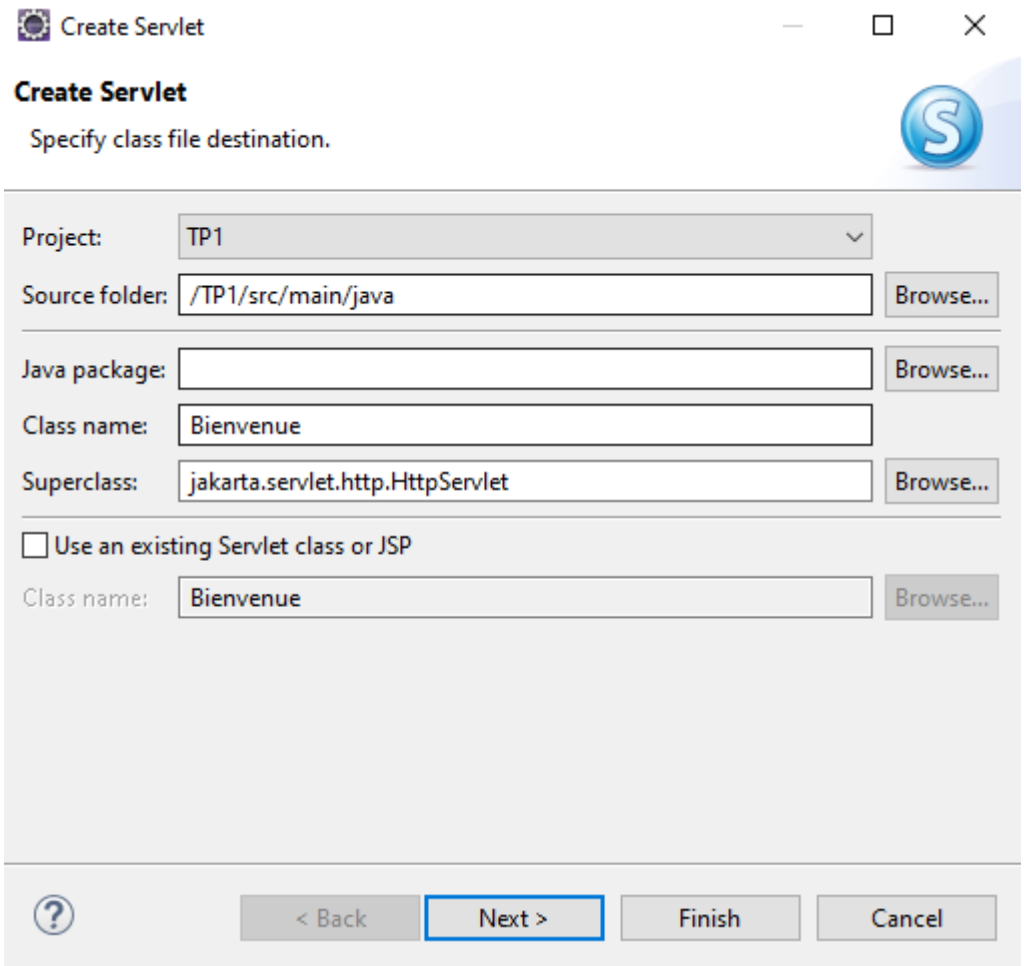




Exercice 1 :

1.Écrivez un programme de servlet pour afficher le message **« Bienvenue les étudiants de la Filière SMI dans la première séance des TPs »** dans le navigateur en utilisant web.xml pour mapper la servlet.

Créations de Servlet Bienvenue :



Code :

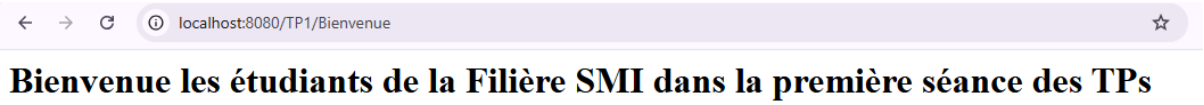
→ *Fichier .java*

```java
import jakarta.servlet.ServletException;

public class Bienvenue extends HttpServlet {
    private static final long serialVersionUID = 1L;

    public Bienvenue() {
        super();
        // TODO Auto-generated constructor stub
    }

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOExceptior

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        out.println("<h1>Bienvenue les étudiants de la Filière SMI dans la première séance des TPs</h1>");
    }

}
```

→ Fichier web.xml

```
  <servlet>
    <description></description>
    <display-name>Bienvenue</display-name>
    <servlet-name>Bienvenue</servlet-name>
    <servlet-class>Bienvenue</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>Bienvenue</servlet-name>
    <url-pattern>/Bienvenue</url-pattern>
  </servlet-mapping>
</web-app>
```

Résultat :



2.Concevez un formulaire HTML pour invoquer une servlet qui devrait attraper l'adresse, le prénom et le nom, et affichez-les côté client au format HTML. (Utiliser les annotations pour le déploiement de la servlet)

Code :

→ Fichier .html:

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Formulaire Utilisateur</title>
</head>
<body>

    <h2>Formulaire d'inscription</h2>

    <form action="Bienvenue" method="post">

        <label for="prenom">Prénom :</label><br>
        <input type="text" id="prenom" name="prenom" required><br><br>

        <label for="nom">Nom :</label><br>
        <input type="text" id="nom" name="nom" required><br><br>

        <label for="adresse">Adresse :</label><br>
        <input type="text" id="adresse" name="adresse" required><br><br>

        <input type="submit" value="Envoyer">

    </form>
</body>
</html>
```

→ Fichier .java

```java
import jakarta.servlet.ServletException;

@WebServlet(value="/Formulaire")

public class Formulaire extends HttpServlet {
    private static final long serialVersionUID = 1L;

    public Formulaire() {
        super();

    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {

        response.setContentType("text/html");

        String prenom = request.getParameter("prenom");
        String nom = request.getParameter("nom");
        String adresse = request.getParameter("adresse");


        PrintWriter out = response.getWriter();
        out.println("<h1>Bienvenue , </h1>");
        out.println("<p><i>"+prenom+" "+nom+"</i>"+" "+"<b>"+adresse+"</b></p>");

    }

}
```

Résultat :

Formulaire d'inscription

Prénom :
madani

Nom :
karim

Adresse :
DR AHEL SABEK FOUM EL

Envoyer

Bienvenue ,

karim madani **DR AHEL SABEK FOUM EL ANSSER BENI MELLAL**

Exercice 2 :

Créer une page HTML contenant trois champs, le premier champ permet de saisir le premier entier, le deuxième champ permet de choisir l'opération que l'utilisateur désire effectuer et le troisième champ permet de saisir le deuxième entier. Les données seront envoyées après à une servlet qui affiche le résultat final.

Code :

→ Fichier .java

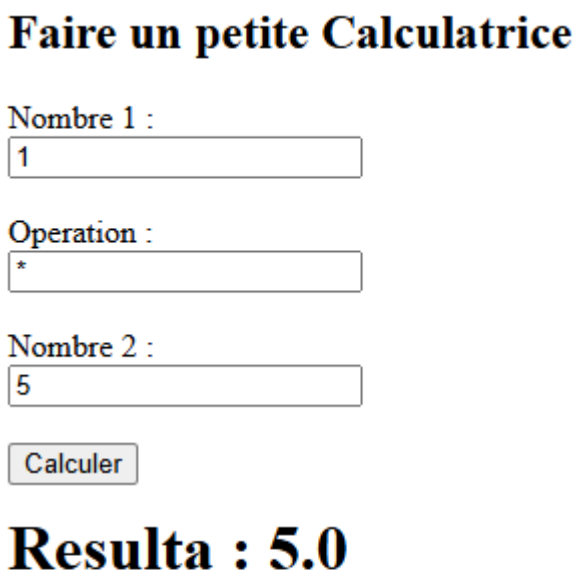
```java
import jakarta.servlet.ServletException;

@WebServlet(value = "/calculatrice")
public class calculatrice extends HttpServlet {
    private static final long serialVersionUID = 1L;

    public  calculatrice() {
        super();
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {

        response.setContentType("text/html");

        //Pour le programme soit faible recommande d'ajouter bloc (try - cache)
        double n1 = Double.parseDouble(request.getParameter("n1"));
        double n2 = Double.parseDouble(request.getParameter("n2"));
        String o = request.getParameter("o");

        PrintWriter out = response.getWriter();
        out.println("<h1>Resulta : "+calcule(n1,n2,o)+"</h1>");

    }
```

```java
public static String calcule(double n1,double n2 ,String o) {
    if(o.equals("*")) {
        return String.valueOf(n1*n2);
    }
    if(o.equals("+")) {
        return String.valueOf(n1+n2);
    }
    if(o.equals("-")) {
        return String.valueOf(n1-n2);
    }
    if(o.equals("/")) {
        if(n2==0) {
            return "Erreur tu ne peux pas diviser sur 0.";
        }
        return String.valueOf(n1/n2);
    }
    return "Soit le premier nombre , deuxieme nombre , operations invalide(seulment * + - / ) ";

}
```

→ Fichier .html

```html
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Mini Calculatrice</title>
</head>
<body>

    <h2>Faire un petite Calculatrice</h2>

    <form action="calculatrice" method="post">

        <label for="n1">Nombre 1 :</label><br>
        <input type="text" id="n1" name="n1" required><br><br>

        <label for="o">Operation :</label><br>
        <input type="text" id="o" name="o" required><br><br>

        <label for="n2">Nombre 2 :</label><br>
        <input type="text" id="n2" name="n2" required><br><br>

        <input type="submit" value="Calculer">

    </form>
</body>
</html>
```

Résultat :



Exercice 3 :

Code :

→ Code .java

```java
import jakarta.servlet.ServletException;

@WebServlet(value = "/temps")

public class Temps extends HttpServlet {
    private static final long serialVersionUID = 1L;

    public Temps() {
        super();
    }


    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws Servl

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        // Obtenir la date et l'heure actuelles
        LocalDateTime maintenant = LocalDateTime.now();

        // Définir un format pour l'affichage
        DateTimeFormatter format = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

        out.println("<h1>"+maintenant.format(format)+"</h1>");
        response.setIntHeader("Refresh", 0);
    }

```

Résultat :

Conclusion du TP JEE

Ce TP nous a permis de **découvrir et de manipuler les composants essentiels de Java EE**, notamment les **Servlets**, et la gestion des **requêtes et réponses HTTP**.

Nous avons appris à :

- Créer et configurer une **Servlet** pour traiter les données d'un formulaire.

Grâce à ce TP, le fonctionnement d'une application web Java et l'importance de l'architecture client-serveur. Ces notions constituent une base solide pour développer des applications web **robustes et interactives** avec Java EE.

Technologies utilisées

Pour réaliser ce TP JEE, nous avons utilisé plusieurs outils et technologies qui facilitent le développement et le test des applications web :

1. **Eclipse IDE** :
   Eclipse est un environnement de développement intégré (IDE) très populaire pour Java. Il offre des fonctionnalités comme l'édition de code, la complétion automatique, la gestion de projets JEE et l'intégration avec les serveurs d'applications. Eclipse permet de développer les Servlets, les JSP et de gérer facilement le déploiement sur le serveur.
2. **Google Chrome** :
   Google Chrome est un navigateur web utilisé pour **tester et visualiser l'application web**. Il permet de vérifier l'affichage des pages JSP et de simuler l'interaction avec l'utilisateur en envoyant des requêtes HTTP vers les Servlets.
3. **Apache Tomcat** :
   Tomcat est un serveur d'applications web open-source qui prend en charge les technologies Java EE comme les **Servlets et JSP**. Il permet d'exécuter et de tester l'application développée dans Eclipse. Tomcat gère les requêtes HTTP, exécute le code Java côté serveur et renvoie les réponses au navigateur.